DataChat

DCC-6114

Detailed Design Document

Heeyoon Shin, Brayden Huguenard, Gil Hananel, Terrance Hollingsworth, Henry Arnold

Client: Martin Rojas

Repository: https://github.com/gilel6/PlayOnProject

Table of Contents

List of Figures *3*

Terminology *4*

Introduction *6*

Background **6**

Document Summary **6**

System Architecture *7*

Introduction **7**

Rationale **7**

Frontend Framework: ReactJS and Tailwind CSS 7

Backend and AI: Python/FastAPI and OpenAI 7

Data Management: PostgreSQL 8

Static System Architecture **8**

Dynamic System Architecture **9**

Data Design *11*

Introduction **11**

Database Use **12**

Diagram 12

File Use **12**

Data Exchange **13**

Security Considerations 13

UI Design Walkthrough *14*

Introduction **14**

Appendix *19*

List of Figures

Figure 1 - Static System Architecture 8
Figure 2 - DataChat Sequence Diagram 9
Figure 3 - Runtime Interactions 10
Figure 4 – Data Design Diagram 12
Figure 5 - DataChat Login Screen 14
Figure 6 - DataChat Main Chat Screen 15
Figure 7 - DataChat Prompt Response 16
Figure 8 - DataChat Settings Page 17
Figure 9 - DataChat Archived 18

Terminology

Term	Definition	Context
API	A set of rules or protocols that enable software applications to communicate with each other to exchange data, features, and functionality.	Used to connect frontend and backend, as well as LLMs.
Artificial Intelligence (AI)	The simulation of human-like intelligence by machines, enabling them to perceive, reason, learn, and make decisions based on data.	Our chatbot's core capability.
Chatbot	An interface where users submit a prompt and receive a tailored response. Functionality is highly variable even with the same prompt.	ChatGPT, Gemini, etc.
CSV	A plain-text file format that stores tabular data with values separated by commas.	Used in Excel, Google Sheets; primary upload format for DataChat.
FastAPI	A modern, high-performance web framework for building HTTP-based APIs in Python.	Used to build the DataChat backend.
Hash	A cryptographic function that converts input data of any size into a fixed-size, irreversible string of characters.	Used to encrypt user passwords stored in the database.
Large Language Model (LLM)	An AI model trained on massive text datasets to understand and generate human-like language, enabling tasks such as text completion, summarization, translation, and question answering.	Used via OpenAI to convert natural language to SQL and generate summaries.
OpenAI	A company providing access to large language models (e.g., GPT-5.4) via a REST API.	Used as the AI provider for natural language and SQL generation.
Portable Document Format (PDF)	A file format developed by Adobe that preserves layout, fonts, and graphics across platforms.	Used for exporting chat histories and analysis results.
PostgreSQL	An advanced, open-source relational database that supports both SQL (relational) and JSON (non-relational) querying.	Used to store user credentials, chat sessions, messages, and file metadata.

Primary Key	A unique identifier for each record in a database table.	Used throughout the Data Design section.
Python	A high-level, interpreted programming language known for readability, simplicity, and versatility.	Used for all backend functionality.
React / ReactJS	A JavaScript library for building user interfaces, particularly single-page applications, using a component-based architecture and reactive data flow.	Used for the DataChat frontend.
Relational Database	A database organized into tables with rows and columns, linked by defined relationships.	Implemented via PostgreSQL.
Single-Page Application (SPA)	A web application that loads a single HTML page and dynamically updates content using JavaScript, avoiding full page reloads.	The architecture pattern used for the DataChat frontend.
SQL	Structured Query Language, a standard language for querying and manipulating relational databases.	Generated by the LLM from natural language user input.
Supabase	An open-source Firebase alternative providing a PostgreSQL database, authentication, and storage as a managed cloud service.	Used for persistent chat history and file storage.

Introduction

Background

DataChat is a chatbot application designed for non-technical financial professionals at PlayOn Sports. The primary purpose of the application is to ingest large CSV files — potentially hundreds of thousands of lines — and allow users to query that data using plain natural language. The backend translates those queries into SQL, executes them against the uploaded data, and returns both a plain-English summary and an interactive chart.

Users can create and manage multiple independent chat sessions and tie it to the central database of uploaded files. The application enforces account-level isolation: each user logs in to an account only they can access, and all chats/messages are private to that account. The uploaded CSV files are combined into a central table that can be referenced by month and year from any account registered into the application.

The frontend is built with React and Tailwind CSS. The backend uses Python with FastAPI and integrates OpenAI for natural language processing. Supabase provides PostgreSQL-backed persistent storage for user credentials, chat sessions, messages, and file metadata.

Document Summary

The Static System Architecture section describes the tech stack and the structural relationships between the frontend, backend, AI layer, and databases. It explains why each technology was selected and how the components interact at a high level.

The Dynamic System Architecture section details the runtime behavior of the system across two key flows: the three-step AI pipeline (SQL generation, query execution, and chart generation) triggered by a user message, and the broader session lifecycle from application open through CSV upload, chat, and report generation.

The Data Design section covers the PostgreSQL schema used for user accounts, chat sessions, messages, and uploaded files. It also describes file handling, data exchange mechanisms, and security considerations including password hashing, HTTPS communication, and per-user data isolation.

The UI Design section describes each primary screen of the application (the login screen, main chat interface, and settings pages) with reference to the heuristics applied in their design.

System Architecture

Introduction

The DataChat system is designed to provide a high-performance, intuitive interface for non-technical financial professionals to analyze massive datasets. The primary objectives are:

- **Natural Language Data Extraction:** Enable users to query complex financial data using plain-language prompts, which are converted to SQL for execution.
- **Massive Dataset Processing:** Efficiently process and analyze CSV files that frequently exceed 600,000 lines, reducing manual review time significantly.
- **Automated Insight Generation:** Provide both textual summaries and high-fidelity chart visualizations that explain trends in the underlying data.
- **Reliability and Trust:** Ensure high output accuracy, directly addressing the core failure modes of prior AI solutions used by the client.
- **Efficient Knowledge Sharing:** Support one-click report export and persistent chat history so insights can be shared and revisited.

Rationale

The team has selected a modern, scalable tech stack to balance user-friendly frontend performance with robust, secure backend processing.

Frontend Framework: ReactJS and Tailwind CSS

ReactJS provides a responsive, component-based single-page application architecture well-suited to a real-time chat interface. Tailwind CSS gives the team granular control over styling without the overhead of a component library, keeping the UI uncluttered and accessible for non-technical users.

Backend and AI: Python/FastAPI and OpenAI

FastAPI serves as the central request router, handling user input and dispatching to the AI layer and databases with minimal latency. OpenAI provides access to GPT models required for natural language to SQL conversion and financial data reasoning. Python's library ecosystem supports the additional data processing requirements of the use case.

Data Management: PostgreSQL

Supabase provides a managed PostgreSQL instance with built-in authentication and storage. The relational model ensures chat history, user credentials, and file metadata are organized and easily retrievable. Using Supabase also simplifies deployment by consolidating database, auth, and file storage into a single managed platform.

Static System Architecture

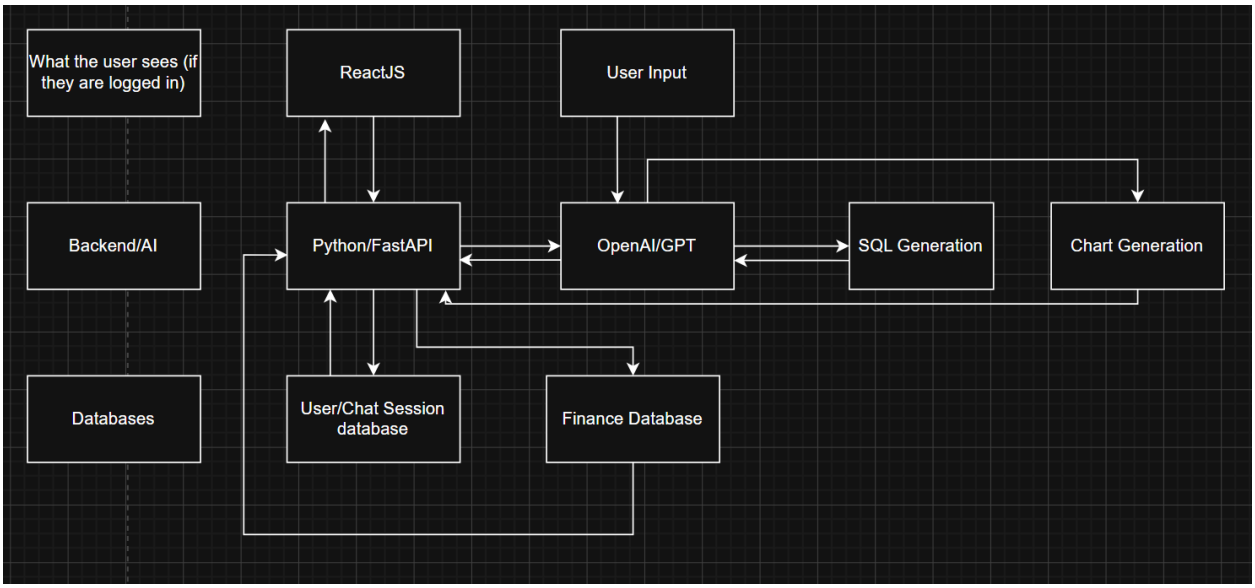


Figure 1 - Static System Architecture

Figure 1 illustrates the structural relationships between the major system components. The React frontend communicates with the FastAPI backend via JSON over HTTPS. The backend interfaces with Supabase (PostgreSQL) for persistent storage and with OpenAI for AI processing. User input enters through the frontend and drives all downstream interactions.

Dynamic System Architecture

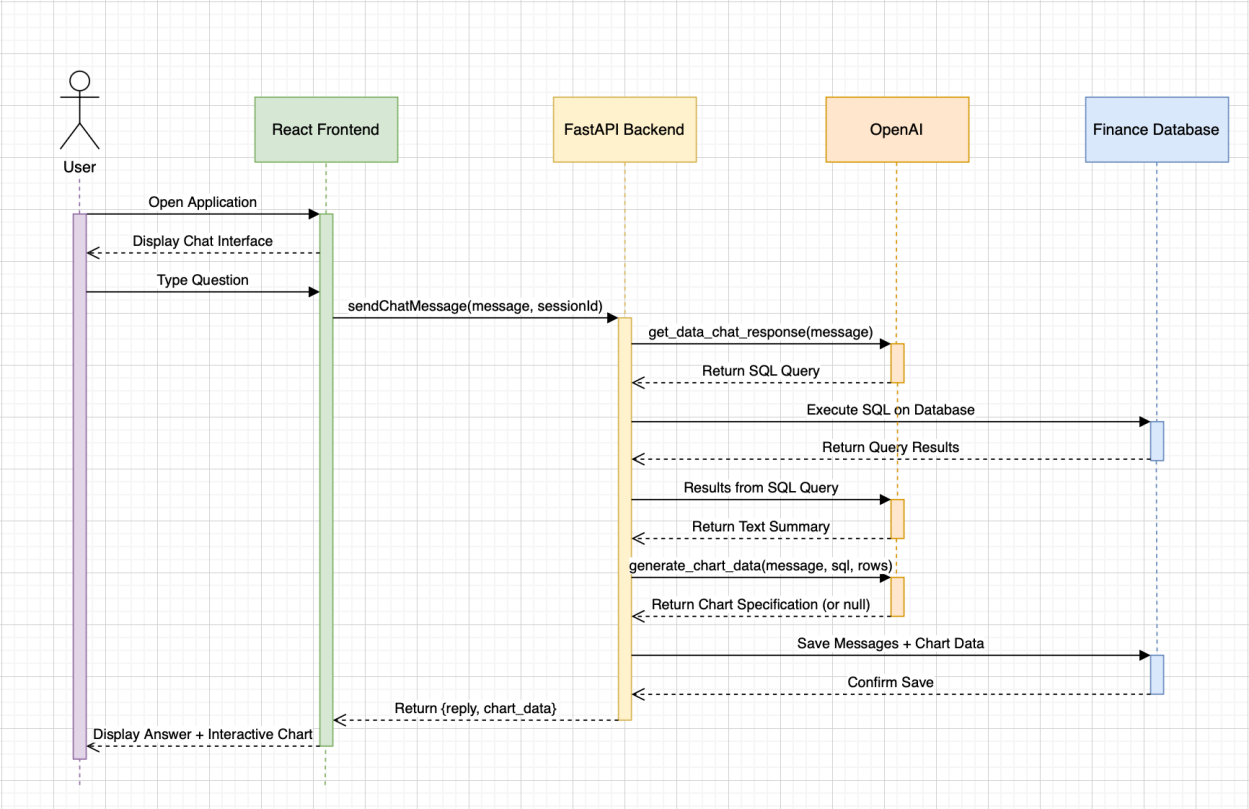


Figure 2 - DataChat Sequence Diagram

The functionality begins when the user opens the application and the React frontend renders the chat interface. Once the user types a question, the frontend calls sendChatMessage(message, sessionId) to the FastAPI backend, initiating a three-step AI pipeline.

First, we start with the SQL Generation. The backend calls get_data_chat_response(message), forwarding the user's natural language question along with the database schema to OpenAI. GPT returns a SQL query tailored to the user's financial data.

Next, we execution the query and summarize the results. The backend executes that SQL against the Finance Database in read-only mode and receives the raw results. Those results are then sent back to OpenAI with the original question, and GPT returns a plain-English summary of the findings.

Finally, we generate charts to support digestibility of the data. The backend makes a final call to OpenAI via generate_chart_data(message, sql, rows), passing the original question, the SQL, and the result rows. GPT returns a chart specification JSON describing the visualization type and data mapping, or null if no chart is appropriate.

Persistence & Response The backend saves both the assistant message and the chart data to the database, receives confirmation, then returns {reply, chart_data} to the frontend. The frontend renders the text answer alongside an interactive chart.

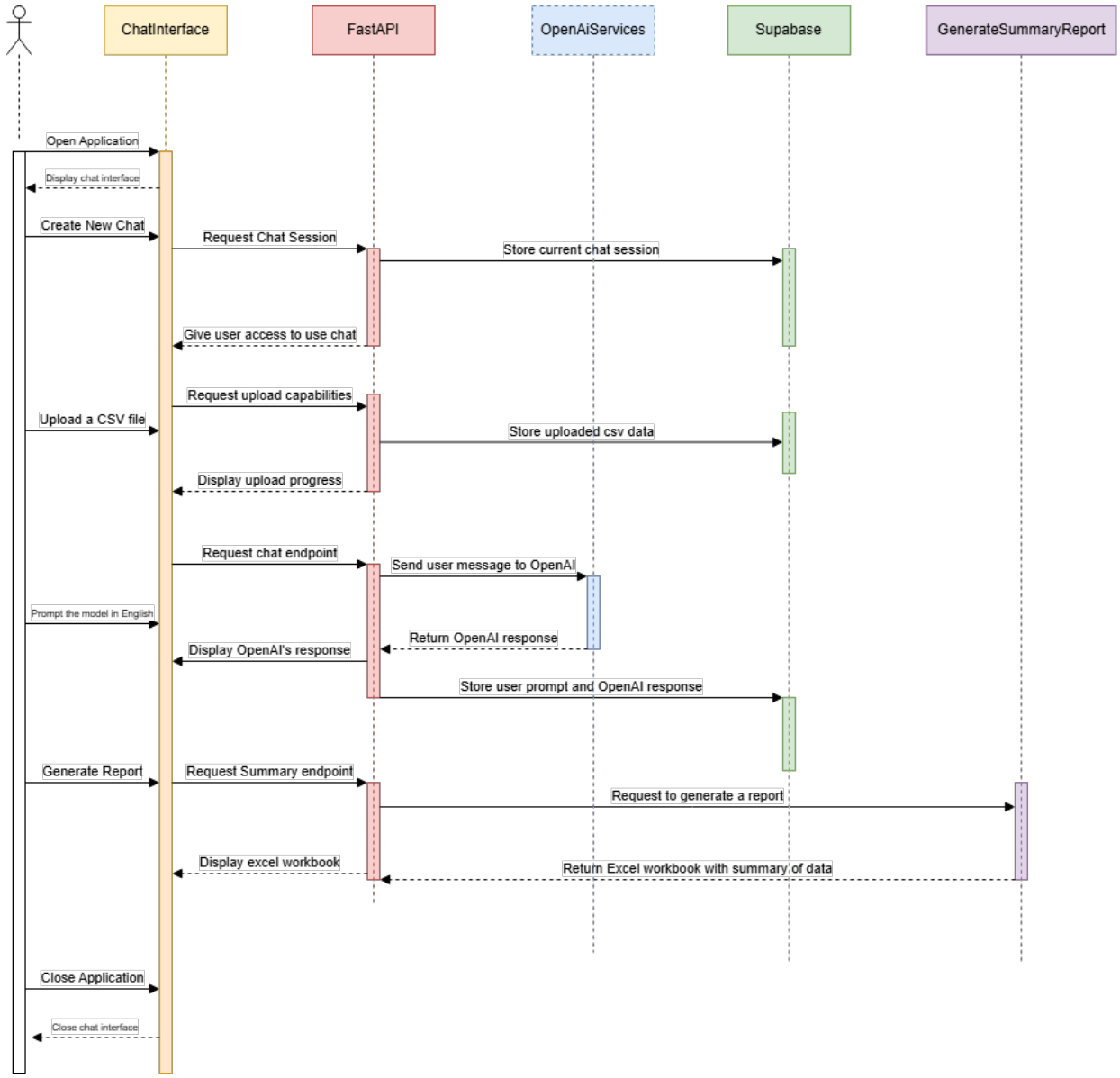


Figure 3 - Runtime Interactions

The interactions at runtime begin when the user opens the application, the user is displayed the initial chat interface, as previously mentioned. Once the user creates a new chat, a request will be sent to our backend API for a chat session. After the user is

authorized, they will be given the ability to use the chat AI features and upload features. Additionally, the chat session will be stored in Supabase, allowing the ability to preserve chat history and information.

When the user uploads a csv file, a request is sent to the backend API to give the user access to the upload capabilities. This event will trigger upload progress to be displayed to the user. The CSV metadata is stored in the files table and the raw data is stored in our database for raw data.

When the user prompts the model, a request is sent to the chat endpoint for our backend FastAPI. The user's messages are then sent to our custom OpenAI services and fed into the prompt for our text-to-sql model. The model's answer is then returned to the backend API and displayed to the user. The metadata for each chat message (user/bot) is stored in Supabase.

Finally, when the user clicks the "Generate Report" button a request is sent to the summary API endpoint. This triggers the generate report function to run and the excel workbook is returned to the summary endpoint. The workbook is then displayed to the user.

Data Design

Introduction

For this application, there will be two separate database components. The first database component will be a PostgreSQL database that contains tables related to user account information, file information, and chat sessions. The user account table will contain information such as user id (primary key), email, password hash, the time the account was created, and if the account is active. The chat sessions table will contain information such as chat id, user id, title of the chat, the timestamp the chat was created, the last time a message was sent, and if the chat is pinned or not. The file table will contain information such as the file id, chat id, the name of the file, the storage path, a file hash, file type, file size, user id, and upload timestamp. This database component is what will enable the user to have a chat history that persists.

Database Use

Diagram

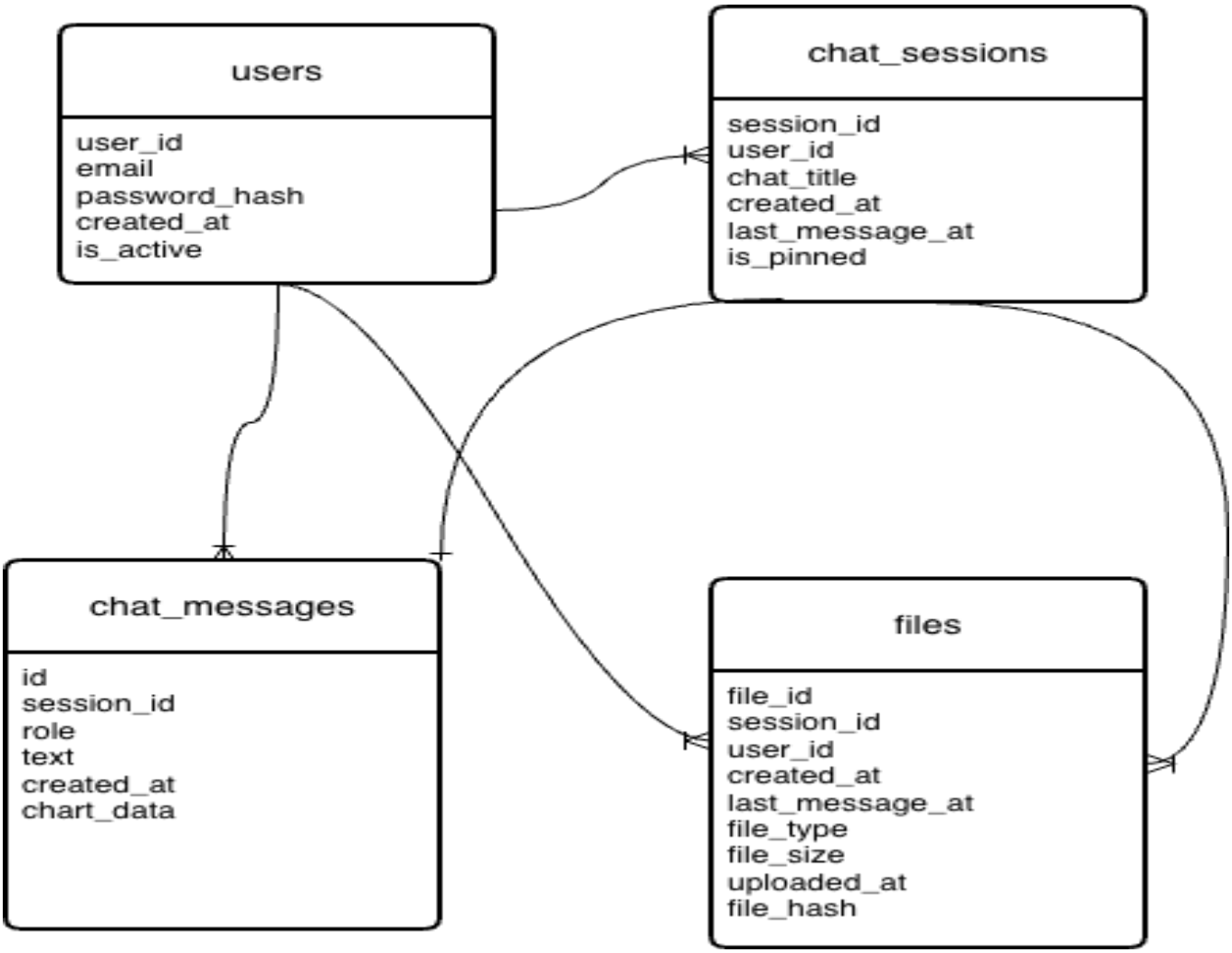


Figure 4 – Data Design Diagram

File Use

This application is heavily dependent on CSV file analysis. The application must be able to analyze the CSV files so that users can get results they can read. The application cannot just summarize the CSV files, it must give out reliable comparisons that are beneficial for the users using this application. The application, while less important than CSV files, should also be able to support non-CSV files as well. The analysis will include visualizations in the form of exportable PDF files. The charts and chats themselves will be exportable in this format.

Data Exchange

Our web client will have the ability to login, upload CSV files, and chat with our NL to SQL bot. The React frontend we use will directly interact with the FastAPI/Python backend. The data uses JSON requests/responses over HTTPS protocol for security. Our backend will interact with our PostgreSQL database for the SQL queries and relational table responses. This is where user credentials/information and chat metadata will be stored. It will also be interacting with our PostgreSQL database for the CSV data, and OpenAI with JSON requests/responses with the LLM over HTTPS. This is where the NL is converted to SQL queries.

Security Considerations

Our backend server is hosted within a secure cloud environment and communicates with other components (PostgreSQL and OpenAI) over encrypted HTTPS channels. All communication between the React frontend and FastAPI backend is conducted over HTTPS, ensuring that user inputs, uploaded datasets, and generated insights cannot be intercepted. All uploaded financial datasets are stored in PostgresSQL, which provides encryption at rest and controlled access through secure policies. Access to these files is restricted to authorized users, and files are not publicly accessible. User authentication is required to access the system. User credentials are securely stored in PostgreSQL using hashed passwords, ensuring that sensitive login information is never stored in plain text. Each user operates within an isolated context, meaning that datasets, queries, and chat histories are only accessible to the user who created them. This prevents unauthorized access or data leakage between users. Because the application processes financial data and potentially personally identifiable information (PII), additional safeguards are implemented. Only the minimum necessary data (such as schema or column names) is shared with OpenAI for natural language processing, reducing exposure of sensitive raw data.

UI Design Walkthrough

Introduction

Our design for the user interface revolved around the key features, needs, and user stories we discovered during our user research with the client. As our application acts as a specialized chat interface for PlayOn Sports and features uploading CSV files, we decided to take inspirations from the most popular chatbots, including ChatGPT which similarly features file uploading.

When the user first directs the website, they are greeted to the account screen, where they must either create a new account or login with an existing one.

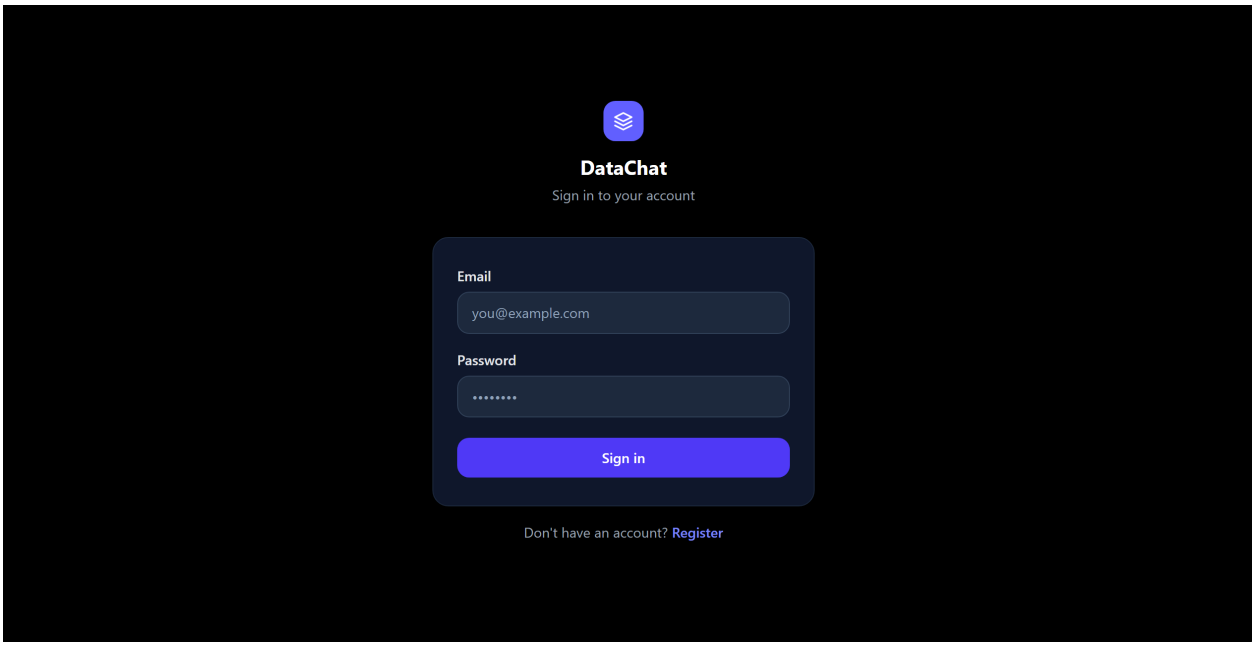


Figure 5 - DataChat Login Screen

The account screen utilizes heuristics including Visibility of System Status, Aesthetic and Minimalistic Design, and Helping User Recognize, Diagnose, and Recover From Error through various design factors. We maintained a minimalistic and standard login screen which clearly conveys what action the user needs to take, along with direct popup errors which explain any input issues with their response.

Once they login, they are greeted with the applications central page. They have the option to either add a new chat, manage/navigate existing chats, continue speaking in a currently selected chat, or go to the settings page in the bottom left corner (as indicated by the gear).

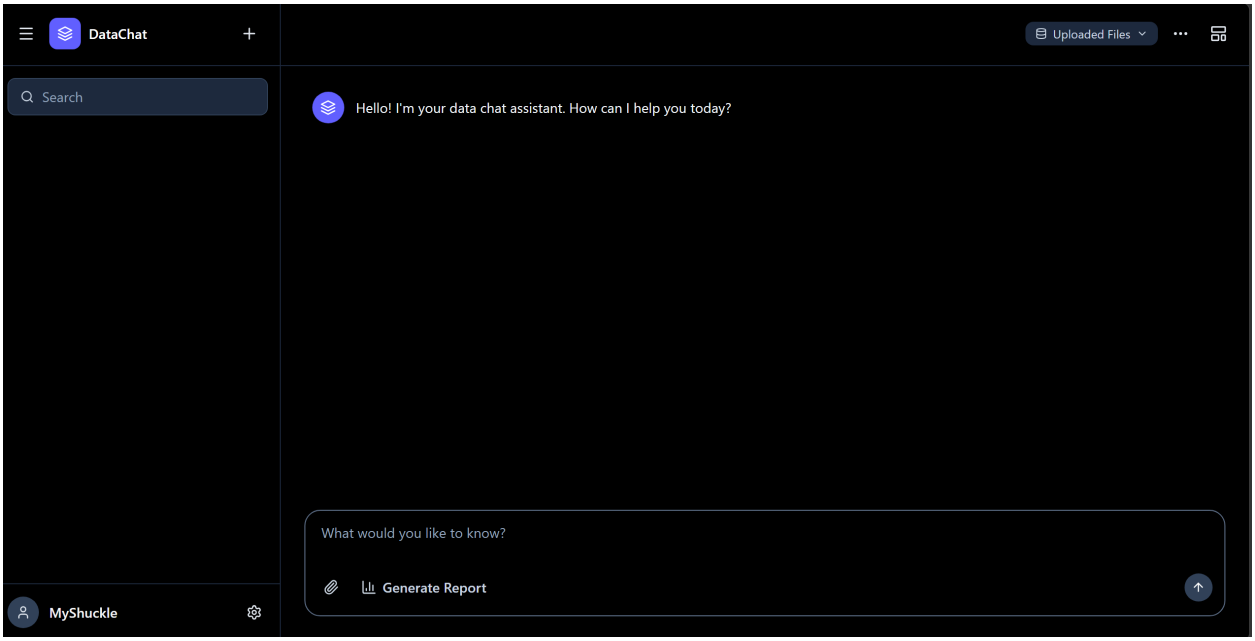
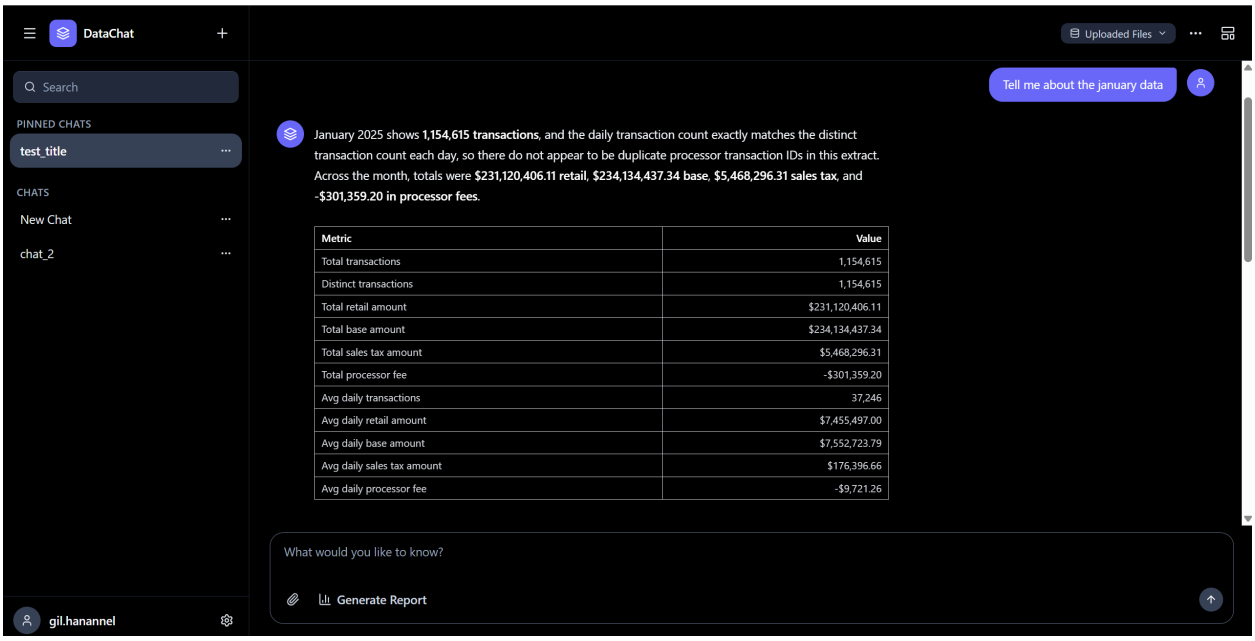


Figure 6 - DataChat Main Chat Screen

Our main chat screen integrates Recognition Rather Than Recall, User Control and Freedom, and Match Between System and the Real World heuristics. We provide the user with an array of options visible on screen which allows them to interact with chats or any part of the interface they need. Additionally, we decided to follow common chatbot design standards to make the interface instantly understandable to users.

Users can ask the chatbot any text prompt due to the OpenAI integration, but the main use of the application is for prompting sports data analysis based off uploaded CSV files. With the generation of a new chat, users can choose to archive or permanently delete chats, come back to previous chats on the left sidebar, store the entire chat history as a txt or pdf file, and view analytics of the current chat.



Metric	Value
Total transactions	1,154,615
Distinct transactions	1,154,615
Total retail amount	$231,120,406.11
Total base amount	$234,134,437.34
Total sales tax amount	$5,468,296.31
Total processor fee	-$301,359.20
Avg daily transactions	37,246
Avg daily retail amount	$7,455,497.00
Avg daily base amount	$7,552,723.79
Avg daily sales tax amount	$176,396.66
Avg daily processor fee	-$9,721.26

Figure 7 - DataChat Prompt Response

These chat conversations were designed with Match Between the System and the Real world, Recognition Rather than Recall, and Aesthetic and Minimalist Design, heuristics in mind. The bot responds in the same language as the user, using phrasing that matches the context. In addition, important terms and figures are bolded to draw the users attention and ensure reading is easy, while charts are sent to easily display the important data and its labels. Finally, the entire response only displays what is needed for the conversation, ensuring the design isn't cluttered.

When the user clicks on the settings page, they are greeted with multiple options: A way to change their profile information, account features, an option to change their appearance from light to dark, or vice versa (the current screenshots are done in dark mode), and the option to see what chats are already archived previously, with the option to restore said chats.

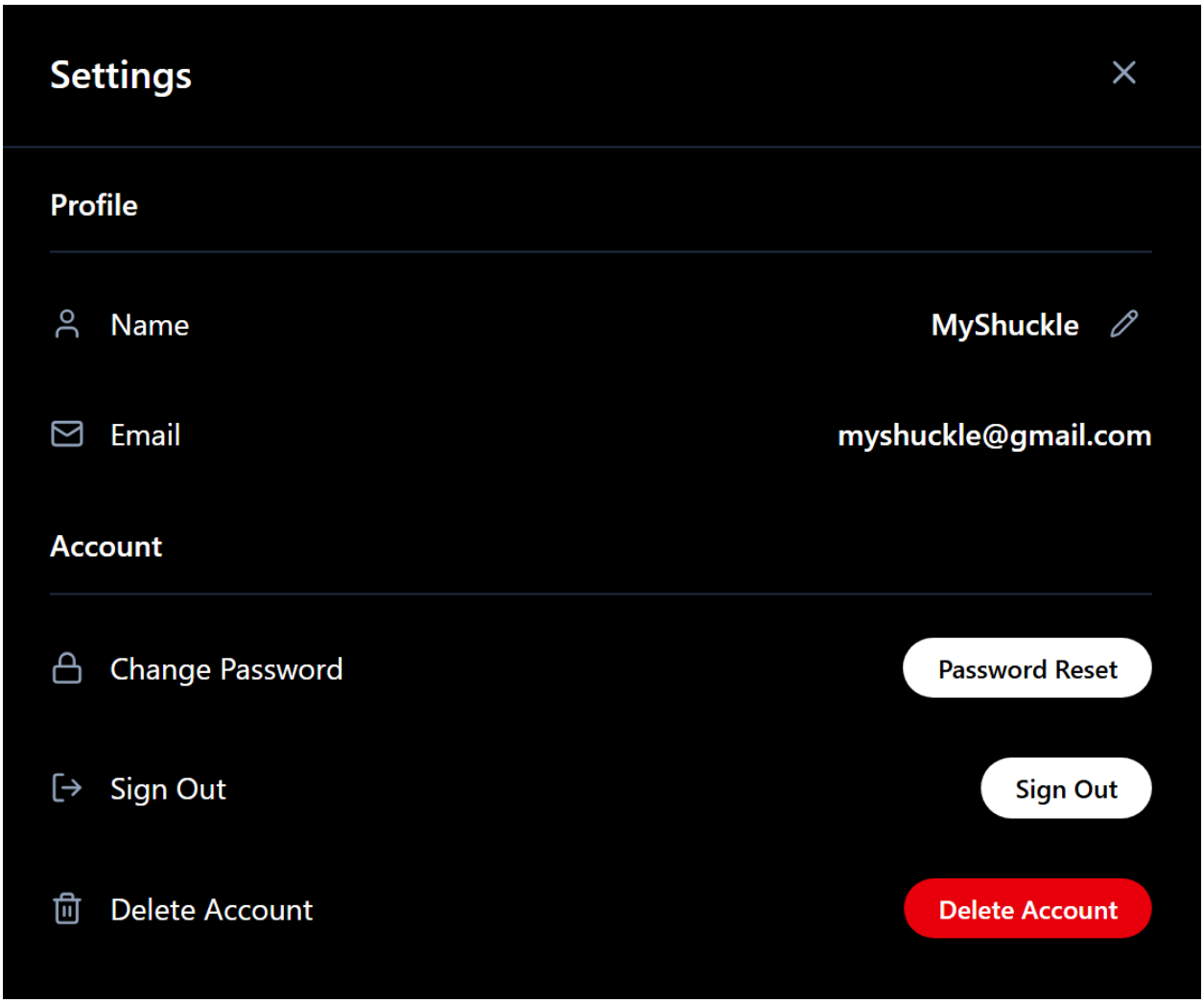


Figure 8 - DataChat Settings Page

The settings page utilizes Visibility of system status, User Control and Freedom, and Aesthetic and Minimalist Design heuristics. We maintained a clear design which displays the state of important to know settings information, along with displaying all available settings related actions.

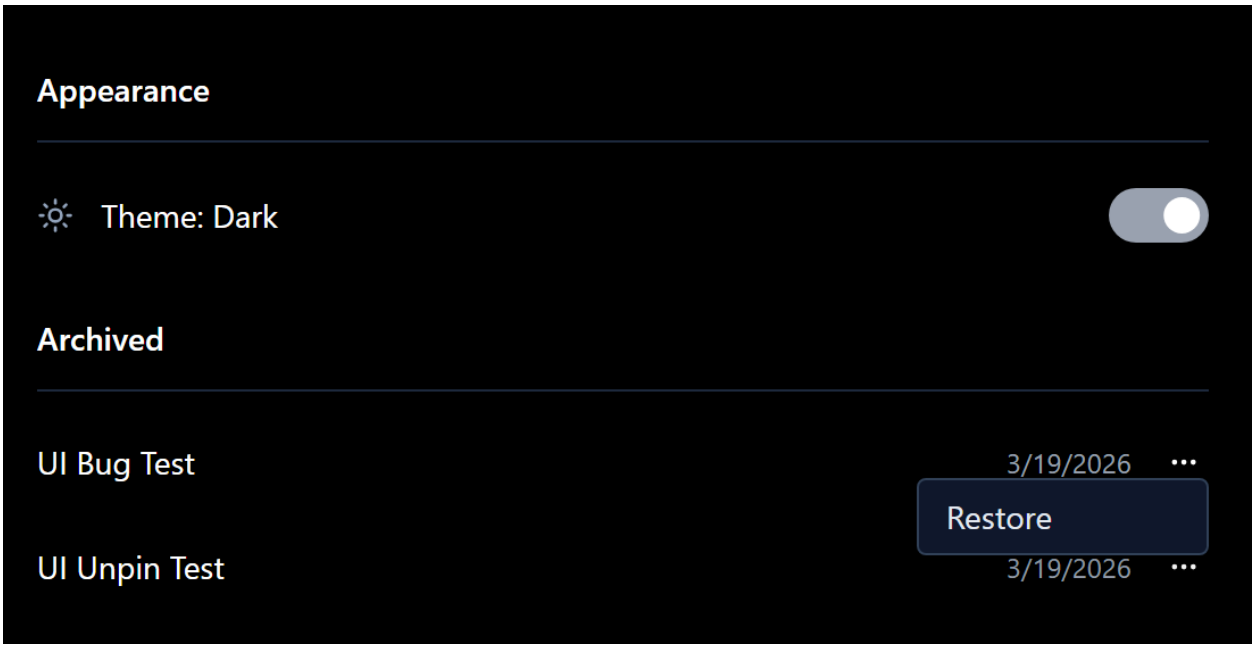


Figure 9 - DataChat Archived

The settings submenu integrates Visibility of System Status, User Control and Freedom, and Aesthetic and Minimalist Design heuristics. As this menu acts as an extension of the settings page, we wanted to continue our focus on making it clear of clutter, easy to read the current state of your theme or archived chats, and an easy to use through commonly used UI components.

Appendix

Members:

Name	Contributions	Contact
Brayden Huguenard	Dynamic System Architecture, Terminology, Table of Figures	bhuguenard3@gatech.edu
Gil Hananel	Data Design, UI Design	ghananel3@gatech.edu
Henry Arnold	Data Design, Introduction	Harnold9@gatech.edu
Terrance Hollingswroth	System Architecture	hollingsworthmercer1@gatech.edu
Heeyoon Shin	Static System Architecture, UI Design	Hshin327@gatech.edu